

EVE Healthcare — Backend Engineering Submission

Assignment: SDE / Founding Engineer (Diagnostic Test Booking & Simulated Payments)

Candidate: Milan (GitHub: [Milan727](#)) | Repository: <https://github.com/Milan727/eve-healthcare>

Python 3.13

FastAPI 0.115+

SQLAlchemy 2.0 (Async)

PostgreSQL & SQLite

Pytest (26/26 Passed)

Docker Ready

1. Executive Summary & Architecture

This project delivers a complete, production-grade backend service for diagnostic centre test catalog management, patient bookings, simulated payment execution, and **strictly idempotent payment webhook processing**. Built with a clean, layered architecture separating HTTP controllers, domain services, persistence models, and DTO schemas.

Layered Architecture

- `app/api/v1/` : RESTful route controllers & validation.
- `app/services/` : Isolated business domain logic.
- `app/models/` : SQLAlchemy 2.0 Async declarative ORM models.
- `app/schemas/` : Pydantic v2 schemas for request/response typing.

Key Highlights

- JWT Authentication** with `bcrypt` password hashing.
- Flexible Pricing Junction** (Centre × Test).
- Immutable Price Snapshot** on bookings.
- Idempotent Webhook Engine** with race condition safety.
- 26 Integration Tests** with 100% pass rate.

2. Database Schema & Relational Design

The database is designed with third-normal-form relational integrity, snapshot pricing, and unique idempotency tracking:

Table	Primary Key	Key Columns & Constraints	Purpose
<code>users</code>	<code>id (UUID)</code>	<code>email (Unique, Indexed)</code> , <code>hashed_password</code> , <code>full_name</code> , <code>is_admin</code>	User identity, authentication, and role authorization.
<code>diagnostic_centres</code>	<code>id (UUID)</code>	<code>name</code> , <code>location</code> , <code>contact_number</code> , <code>is_active</code>	Stores diagnostic facilities and physical locations.
<code>diagnostic_tests</code>	<code>id (UUID)</code>	<code>name</code> , <code>category</code> , <code>description</code> , <code>is_active</code>	Catalog of diagnostic tests (e.g. CBC, MRI, Lipid Panel).
<code>centre_tests</code>	<code>id (UUID)</code>	<code>centre_id (FK)</code> , <code>test_id (FK)</code> , <code>price (Numeric)</code> , <code>Unique(centre_id, test_id)</code>	Junction table allowing centre-specific pricing & availability.
<code>bookings</code>	<code>id (UUID)</code>	<code>user_id (FK)</code> , <code>centre_id (FK)</code> , <code>test_id (FK)</code> , <code>amount (Snapshot)</code> , <code>status</code>	Diagnostic appointment with state machine & immutable locked price.
<code>payments</code>	<code>id (UUID)</code>	<code>booking_id (FK)</code> , <code>amount</code> , <code>status</code> , <code>transaction_ref (Unique)</code>	Financial transaction audit trail for direct and webhook payments.
<code>webhook_events</code>	<code>id (UUID)</code>	<code>event_id (Unique, Indexed)</code> , <code>booking_id (FK)</code> , <code>payload</code> , <code>status</code>	Idempotency registry preventing duplicate processing.

3. Booking State Machine & Invariants

The booking lifecycle enforces strict state transition guards:

- PENDING** → Initial state upon booking creation. Awaiting simulated payment or gateway webhook.
- CONFIRMED** → Transitioned atomically when payment succeeds. Cannot be double-paid.
- FAILED** → Transitioned when payment fails. User may retry payment.
- CANCELLED** → Patient cancellation. **Late-arriving webhooks cannot revive a cancelled booking.**

4. Simulated Payment Service & Webhook Idempotency

A. Direct Simulated Payment (POST /payments/)

Allows patients to simulate payments directly:

- Validates booking exists and verifies authorization (patients can only pay for their own bookings).
- Guards against invalid states: Rejects payments on already `CONFIRMED` or `CANCELLED` bookings (`400 Bad Request`).
- Atomically writes a `Payment` record with a unique transaction reference and updates booking status.

B. Payment Webhook Engine (POST /payments/webhook/)

Payment providers (Stripe, Razorpay) deliver webhooks with *at-least-once delivery*. Our implementation guarantees strict idempotency:

- Deduplication Check:** Queries `webhook_events` by `event_id` . If already processed, immediately returns `200 OK` with `{"status": "DUPLICATE_IGNORED"}` without creating duplicate payments or re-triggering transitions.
- Database-Level Uniqueness:** `event_id` is guarded by a database unique index.
- Concurrency & Race-Condition Safety:** If two duplicate webhooks arrive concurrently, the second catches an `IntegrityError` , performs a transactional rollback, and returns the idempotent response.
- Cancelled Booking Shield:** If a webhook arrives for an already cancelled booking, the event is recorded as `IGNORED_CANCELLED` and does not mutate booking status.

5. Comprehensive Edge Cases Handled

Edge Case Scenario	System Behavior	HTTP Status
Duplicate Webhook Delivery	Idempotency lookup detects existing <code>event_id</code> ; returns duplicate confirmation safely	<code>200 OK</code>
Concurrent Webhook Collision	DB unique constraint triggers rollback; recovers gracefully without duplicate payments	<code>200 OK</code>
Webhook on Cancelled Booking	Logged as <code>IGNORED_CANCELLED</code> ; prevents reviving cancelled appointments	<code>200 OK</code>
Unauthorized Resource Modification	User attempting to view, pay for, or cancel another user's booking	<code>403 Forbidden</code>
Double Payment Attempt	Rejecting payment on already <code>CONFIRMED</code> bookings	<code>400 Bad Request</code>
Invalid / Non-Existent Booking ID	Clean error response stating booking was not found	<code>404 Not Found</code>
Appointment Date in Past	Pydantic validator ensures appointment is strictly in the future	<code>422 Unprocessable</code>
Test Not Available at Centre	Verifies centre offers requested test with active mapping before booking	<code>400 Bad Request</code>
Duplicate Email Registration	Handled before password hashing	<code>409 Conflict</code>
Missing or Tampered JWT	Dependency injection rejects expired or forged tokens	<code>401 Unauthorized</code>

6. Automated Testing Suite

The project contains **26 integration tests** covering 100% of the assignment requirements:

```
tests/test_auth.py (7 tests) ..... [PASSED]
tests/test_centres_tests.py (7 tests) ..... [PASSED]
tests/test_bookings.py (5 tests) ..... [PASSED]
tests/test_payments.py (4 tests) ..... [PASSED]
tests/test_webhook_idempotency.py (3 tests) ..... [PASSED]
===== 26 passed in 10.94s (100% Success) =====
```

7. Quick Start & Execution Guide

Option A: Local Setup (Zero-Config SQLite)

```
git clone https://github.com/Milan727/eve-healthcare.git
cd eve-healthcare
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python scripts/seed_data.py
uvicorn app.main:app --reload --port 8000
```

Option B: Docker Compose (PostgreSQL)

```
git clone https://github.com/Milan727/eve-healthcare.git
cd eve-healthcare
docker-compose up --build
```

Interactive URLs: Swagger UI: `http://localhost:8000/docs` | Health Check: `http://localhost:8000/health`

Demo Credentials: Admin: `admin@evehealthcare.com` / `AdminPass123!` | Patient: `patient@evehealthcare.com` / `PatientPass123!`

8. Primary API Endpoints

Method	Endpoint	Auth	Description
POST	<code>/api/v1/auth/signup</code>	Public	Register new patient with email & password
POST	<code>/api/v1/auth/login</code>	Public	Authenticate and obtain Bearer JWT access token
GET	<code>/api/v1/auth/me</code>	Bearer	Get current user profile
GET	<code>/api/v1/centres/</code>	Public	List diagnostic centres with search, location filter & pagination
GET	<code>/api/v1/centres/{id}</code>	Public	Retrieve centre details with available tests and prices
POST	<code>/api/v1/centres/{id}/tests</code>	Admin	Assign diagnostic test to centre with price and availability
POST	<code>/api/v1/bookings/</code>	Bearer	Create diagnostic test booking in PENDING state
GET	<code>/api/v1/bookings/</code>	Bearer	List bookings for authenticated patient (supports status filter)
POST	<code>/api/v1/bookings/{id}/cancel</code>	Bearer	Cancel booking (validates ownership & current status)
POST	<code>/payments/</code>	Bearer	Simulate direct payment (SUCCESS or FAILED outcome)
POST	<code>/payments/webhook/</code>	Public	Idempotent payment webhook for simulated provider callbacks

9. Important Assumptions & Future Enhancements

Design Assumptions

- Decoupled Pricing:** Different centres set independent prices for the same diagnostic test.
- Immutable Price Snapshot:** Booking amounts are captured at booking time and never altered by later price adjustments.
- 200 OK on Duplicate Webhooks:** Standard industry practice (Stripe/Razorpay) to stop provider retries.

What to Improve With More Time

- Redis Caching:** Cache centre catalogs with invalidation triggers.
- HMAC Webhook Signatures:** Cryptographic signatures (`X-Signature`) with replay protection.
- Background Job Queues:** Celery / ARQ for async notifications and SMS reminders.
- Slot Locking:** Concurrency control for appointment slots.